КУРС ДЛЯ НАЧИНАЮЩИХ · НИКАКИХ ЗНАНИЙ ЗАРАНЕЕ НЕ НУЖНО

Как создать сайт от А до Я

Понять, как настоящий сайт строится, публикуется и обслуживается каждый день, — открывая, файл за файлом, сайт **The Evergreen Project**.

Сайт: dimogdor.github.io/Evergreen · Исходный код: github.com/Dimogdor/Evergreen

КАК ЧИТАТЬ ЭТОТ КУРС

Каждая глава просто объясняет одну идею, затем показывает, **где её увидеть на сайте** (зелёная рамка «Посмотреть»), а иногда предлагает небольшое **упражнение** (оранжевая рамка). Синие рамки «Сказать вслух» — ключевые фразы для устного объяснения. Запоминать всё не нужно: цель — понять, *кто что делает*.

Содержание

1. Как работает сайт
2. Три языка веба: HTML, CSS, JavaScript
3. Набор инструментов
4. Репозиторий GitHub: папка проекта
5. Astro: «генератор» сайта
6. Отделяем содержимое от кода (YAML)
7. Дизайн на практике: от идеи до экрана
8. Вёрстка и адаптивность (телефон, планшет, компьютер)
9. Фото и видео
10. Сайт на 8 языках
11. Функции, одна за другой
12. Панель администратора (без кода)
13. Маленький сервер: Worker
14. Чтобы Google находил сайт (SEO)
15. Доступность: сайт для всех
16. Безопасность
17. Скорость (производительность)
18. Проверка сайта
19. Публикация: хостинг, домен, DNS
20. Жизненный цикл изменения
21. С чего начать учиться
22. Словарь

СКАЗАТЬ ВО ВСТУПЛЕНИИ

«Сайт — это просто текстовые файлы, которые умеет читать браузер. Всё остальное — инструменты, сервисы — нужно, чтобы писать эти файлы быстрее, делать их красивыми и выкладывать в интернет».

1 · Как работает сайт

Когда вы вводите адрес в браузере (Chrome, Safari, Firefox…), происходят три вещи:

Браузер **→ запрашивает →** Сервер (компьютер, который всегда включён) **→ отправляет файлы →** Браузер их показывает

1. **Адрес** (URL), например `https://dimogdor.github.io/Evergreen/ru/contact/`, говорит, к какому серверу обратиться и какую страницу запросить.
2. **Сервер** отправляет файлы: страницу **HTML**, таблицы стилей **CSS**, скрипты **JavaScript**, изображения.
3. **Браузер** собирает всё вместе и показывает страницу.

Статический или динамический сайт?

	Статический сайт (наш случай)	Динамический сайт (например, WordPress)
Принцип	Страницы собираются **заранее**, один раз, а потом просто отправляются.	Каждая страница собирается **при каждом посещении** программой и базой данных.
Скорость	Очень быстро	Медленнее
Безопасность	Очень мало слабых мест	Частые обновления безопасности
Стоимость	Бесплатный хостинг	Обычно платный сервер

Сайт Evergreen — **статический**: после каждого изменения инструмент за несколько секунд собирает все страницы (их 171!), и они выкладываются на хостинг. Дальше мы увидим, как при этом всё-таки есть панель администратора и формы.

> **ПОСМОТРЕТЬ**
>
> Откройте сайт, правая кнопка мыши → **«Просмотр кода страницы»**: это HTML, который прислал сервер. Затем правая кнопка → **«Просмотреть код»**: это **инструменты разработчика** браузера — лучший друг каждого, кто делает сайты.

2 · Три языка веба

Все сайты мира построены на трёх языках, у каждого своя роль. Простой образ — дом.

Язык	Роль	В доме
HTML	**Структура** и содержимое: заголовки, абзацы, изображения, ссылки, формы.	Стены и комнаты
CSS	**Внешний вид**: цвета, шрифты, размеры, отступы, расположение.	Краска и отделка
JavaScript	**Поведение**: что реагирует на клик, что меняется без перезагрузки.	Электричество, выключатели

HTML: «теги»

```
<h1>Контакты</h1>                      ← большой заголовок
<p>Напишите нам, мы быстро ответим.</p>    ← абзац
<a href="mailto:evergreenecovillage@gmail.com">Написать нам</a>  ← ссылка
<img src="shater.webp" alt="Шатёр в лесу">                    ← изображение + его описание
```

CSS: «правила»

```
h1 { font-family: 'EB Garamond'; color: #22402a; }   ← все большие заголовки тёмно-зелёные
.btn { border-radius: 999px; padding: 0.85em 1.4em; } ← скруглённые кнопки
```

JavaScript: «инструкции»

```
knopka.addEventListener('click', () => {               ← когда нажимают на кнопку…
  navigator.clipboard.writeText('GR7701103910000039100269800');  ← …скопировать IBAN
});
```

ПОСМОТРЕТЬ В РЕПОЗИТОРИИ

- CSS: `src/styles/global.css` — весь стиль сайта в одном файле.
- JavaScript: в конце `src/pages/[lang]/[page].astro`, кнопка «Копировать» для IBAN (строки `<script>`).
- HTML: во всех файлах `.astro` (см. главу 5).

А TypeScript?

Это JavaScript с «пометками», которые указывают тип каждого значения (текст, число…). Он помогает избежать ошибок. На нём написаны файлы `.ts` проекта (например, `src/calendar.ts`).

СКАЗАТЬ ВСЛУХ

«Начни с HTML и CSS: за две недели можно сделать настоящую страницу. JavaScript — потом, когда захочется, чтобы страница реагировала».

3 · Набор инструментов

Инструмент	Для чего	Бесплатно?
Редактор кода: VS Code	Писать и упорядочивать файлы, с подсветкой и подсказками.	Да
Браузер + инструменты разработчика (F12)	Смотреть результат, исследовать, проверять на «ненастоящем» телефоне.	Да
Терминал	Вводить команды (запустить сайт, установить инструменты).	Да
Node.js и **npm**	Node запускает JavaScript на компьютере; npm устанавливает «пакеты» (инструменты, написанные другими).	Да
Git	Хранить историю каждого изменения (как «Отменить», только навсегда).	Да
GitHub	Хранить проект в интернете, работать вместе, запускать автоматическую публикацию.	Да
Figma (по желанию)	Рисовать макеты до программирования.	Да (базовый тариф)

Три команды проекта

```
npm install     ← скачивает инструменты из package.json (один раз)
npm run dev     ← запускает сайт на компьютере: http://localhost:4321 (обновляется при сохранении)
npm run build   ← собирает финальную версию в папку dist/
```

ПОСМОТРЕТЬ

`package.json`: «паспорт» проекта. В нём видны команды (`scripts`) и используемые пакеты (`dependencies`): **astro** (генератор), **@astrojs/sitemap** (карта сайта для Google), **marked** (превращает простой текст в HTML), **yaml** (читает файлы с содержимым) и два шрифта (**Inter** и **EB Garamond**).
`.node-version`: нужная версия Node (22).

Git в 5 словах

- **Репозиторий**: папка проекта со всей её историей.
- **Коммит**: «снимок» файлов в какой-то момент, с сообщением («Inhabitants: new photo of our dogs»).
- **Ветка**: параллельная рабочая копия, чтобы пробовать, не ломая официальную версию (`main`).
- **Pull request**: запрос на слияние ветки с `main`, с проверкой.
- **Push / pull**: отправить / получить коммиты на GitHub / с GitHub.

ПОПРОБОВАТЬ

На github.com/Dimogdor/Evergreen нажмите **«Commits»**: видно каждое изменение сайта, кто его сделал и когда, и можно показать, какие именно строки изменились.

4 · Репозиторий: план проекта

Вот как устроены папки. Хороший порядок — половина работы.

```
Evergreen/
├── package.json            ← паспорт, команды, инструменты
├── astro.config.mjs        ← настройки генератора (адрес сайта, безопасность)
├── README.md               ← инструкция к проекту
├── docs/GUIDE-ADMIN.md     ← инструкция для администраторов
├── public/                 ← файлы, которые копируются как есть
│   ├── admin/              ← панель администратора (config.yml, index.html)
│   ├── uploads/videos/     ← видео
│   └── _headers            ← правила безопасности/кэша для хостинга
├── src/                    ← сердце сайта
│   ├── pages/              ← одна страница = один файл (глава 5)
│   ├── layouts/Base.astro  ← общий каркас: шапка, меню, подвал
│   ├── components/         ← повторно используемые блоки (фото, список событий…)
│   ├── styles/global.css   ← весь дизайн
│   ├── content/            ← ВЕСЬ текст сайта, в файлах YAML (глава 6)
│   ├── assets/uploads/     ← исходные фото (оптимизируются автоматически)
│   ├── lib.ts              ← полезные функции (чтение содержимого, ссылки…)
│   ├── i18n.ts             ← языки
│   └── calendar.ts         ← даты, часовые пояса, файлы календаря
├── worker/                 ← маленький сервер (вход в админку, истории, помощь в написании)
├── tests/                  ← автоматические проверки
└── .github/workflows/      ← роботы GitHub: публикация, сжатие видео
```

СКАЗАТЬ ВСЛУХ

«Код (страницы, дизайн) и содержимое (тексты, фото) лежат в разных папках. Те, кто ведёт это место, трогают только содержимое — через админку; код почти никогда не меняется».

5 · Astro: «генератор» сайта

Написать 171 HTML-страницу вручную невозможно (8 языков × около двадцати страниц). Поэтому используется **генератор статических сайтов**: пишется **один шаблон** на каждый тип страницы, а инструмент заполняет его каждым текстом на каждом языке. Здесь это **Astro** (astro.build).

Файл .astro = две части

```
---
// Часть 1, между чёрточками: JavaScript, который готовит данные (выполняется при сборке сайта)
const page = await one('pages', 'contact', lang);
---
<!-- Часть 2: HTML, в который данные вставляются в фигурных скобках -->
<h1>{page.title}</h1>
<p class="lead">{page.intro}</p>
```

Страницы = адреса

В `src/pages/` имя файла становится адресом. Квадратные скобки `[ ]` означают «переменная»:

Файл	Какие адреса получаются
`src/pages/[lang]/index.astro`	/el/, /en/, /ru/… (8 главных страниц)
`src/pages/[lang]/[page].astro`	/ru/contact/, /ru/place/, /ru/support/… (все текстовые страницы)
`src/pages/[lang]/activities/[slug].astro`	/ru/activities/retreats/… (по странице на каждое направление)
`src/pages/[lang]/calendar.astro`	/ru/calendar/
`src/pages/[lang]/gallery.astro`, `stories.astro`	галерея, истории
`src/pages/calendar.ics.ts`	/calendar.ics: файл календаря, а не страница!

Layout: общий каркас

`src/layouts/Base.astro` содержит то, что одинаково на всех страницах: невидимый `<head>` (название вкладки, описание для Google, значок), шапку с логотипом и меню, выбор языка, подвал с контактами. Каждая страница «вставляется» в него через `<Base> … </Base>`.

Компоненты: детали LEGO

Проще всего для понимания: откройте `src/components/Story.astro` (12 строк) — фото, цитата (`<blockquote>`), имя и страна.

Компонент	Что делает
`src/components/Img.astro`	Показывает любое фото, оптимизированное автоматически (глава 9).
`Blocks.astro`	Блоки «заголовок + текст + фото», которые чередуются слева/справа.
`EventList.astro`	Список событий с датами, ценами и кнопкой «Добавить в календарь».
`Story.astro`	Карточка истории.
`Video.astro`	Видео YouTube/Vimeo, которое загружается только по клику.

6 · Отделяем содержимое от кода

Все тексты, даты, цены и пути к фото хранятся в `src/content/`, в формате **YAML**: очень читаемый текстовый формат из строк «ключ: значение» и отступов.

> **ПОСМОТРЕТЬ: СОБЫТИЕ**
>
> `src/content/events/essence-of-yoga-2026.yml`

```
el:                                  ← греческая версия (основной язык)
  start: '2026-11-28'                ← дата начала
  end: '2026-12-05'                  ← дата окончания
  category: retreat
  image: /src/assets/uploads/yin-shala.webp
  link: https://bookretreats.com/r/8-day-essence-of-yoga-…
  title: Essence of Yoga — χειμερινό εντατικό …
  price: 380 € early bird (κράτηση έως 28/10/2026) · 450 € κανονική τιμή
ru:                                  ← по-русски: только то, что переводится
  title: Essence of Yoga — зимний интенсив с Паносом Кацандрисом
  price: 380 € при раннем бронировании (до 28.10.2026) · 450 € обычная цена
```

Папка	Содержимое
`settings/site.yml`	Название, e-mail, телефон, соцсети, IBAN, цвета сайта
`settings/ui.yml`	Мелкие тексты интерфейса (меню, кнопки) на 8 языках
`pages/`	Текст каждой страницы (главная, место, образ жизни…)
`programs/`	Направления (волонтёрство, ретриты…)
`events/`	Календарь: один файл на событие
`inhabitants/`	Обитатели: Катерина, Костас, собаки, кошки, куры…
`albums/`, `testimonials/`, `extra/`	Галерея, истории, страницы, созданные в админке

Как код читает это содержимое? `src/content.config.ts` объявляет папки, а функция `list()` из `src/lib.ts` читает их и выбирает нужный язык.

> **СКАЗАТЬ ВСЛУХ**
>
> «Никакой базы данных: содержимое — это обычные текстовые файлы, разложенные по папкам. Их можно читать, сохранять, откатывать назад. Админка просто редактирует эти файлы за нас».

7 · Дизайн на практике: от идеи до экрана

Шаг 1 — Собрать вдохновение («мудборд»)

Прежде чем программировать, смотрим сайты, которые нравятся, и записываем, *почему*. Для Evergreen: kaleidoscope.media (элегантные крупные заголовки, много воздуха, нумерованные разделы «01 —») и evergreenyogastudios.com (стиль семьи Evergreen). Собираем и материал: логотип, свежие фото, тон текстов.

Шаг 2 — Выбрать палитру

Хватит 4–6 цветов, взятых из самой темы: здесь кремовая бумага, мох, глубокий лес, терракота. Проверяем **контраст** (текст читается на фоне) с помощью инструмента вроде *webaim.org/resources/contrastchecker*.

```
:root {                       ← src/styles/global.css, в самом начале
  --paper: #f4f0e6;           кремовый фон
  --ink: #17231a;             почти чёрный текст с зелёным оттенком
  --moss: #3d673f;            цвет мха
  --moss-deep: #22402a;       лесной зелёный (полоса ESC, подвал)
  --clay: #b0622a;            терракота (акценты)
}
```

Эти «переменные» (`--paper`…) используются везде; изменив одну строку, меняешь весь сайт. Это можно сделать даже из админки (Настройки → Цвета).

Шаг 3 — Выбрать два шрифта

Шрифт с засечками для заголовков (EB Garamond: тёплый, «книжный») и **простой шрифт** для текста (Inter: очень хорошо читается на экране). Они из Google Fonts, но **установлены внутрь проекта** через пакеты `@fontsource-variable/…`: никаких обращений к Google, быстрее и бережнее к приватности.

Шаг 4 — Шкала отступов и размеров

Вместо фиксированных размеров используем `clamp(мин, идеал, макс)`: заголовок растёт вместе с экраном, но никогда не становится слишком маленьким или слишком большим. Пример: `--gutter: clamp(16px, 4vw, 56px)` = боковой отступ.

Шаг 5 — Единые «визуальные компоненты»

Круглые кнопки с разреженными заглавными буквами (`.btn`), маленькие подписи (`.label`), карточки «фото + заголовок» (`.card`), нумерованные горизонтальные линии (`.rule`). Определив их один раз, используем повсюду: сайт выглядит «спроектированным», а не собранным наспех.

Шаг 6 — Макет (по желанию), потом код

В Figma рисуем главную страницу в версии для компьютера и телефона. Затем переводим её в HTML (структура) и CSS (стиль) — и доводим прямо в браузере с помощью инструментов разработчика.

> **ПОПРОБОВАТЬ**
>
> Откройте сайт, F12 → вкладка «Elements», нажмите на `<html>` и замените `--moss-deep` на `#5a2a6e`: весь сайт меняет цвет прямо на глазах (ничего не ломается, достаточно перезагрузить).

8 · Вёрстка и адаптивность

Один и тот же сайт должен хорошо выглядеть и на телефоне (390 px в ширину), и на большом экране (1440 px и больше). Это и есть **адаптивный дизайн** (responsive).

Два инструмента вёрстки современного CSS

Инструмент	Для чего	Пример на сайте
Flexbox	Выстроить элементы в ряд (или в столбец).	Шапка: логотип слева, меню справа (`.site-header`).
Grid	Сетка из строк и столбцов.	Карточки направлений, обитатели (`.grid.cols-3`, `.cols-4`).

```
.cols-3 { grid-template-columns: repeat(auto-fill, minmax(min(100%, 340px), 1fr)); }
← «столько колонок шириной от 340 px, сколько поместится»: 3 на компьютере, 1 на телефоне, больше ничего не нужно

@media (max-width: 760px) {                          ← «если экран уже 760 px…»
  .event { grid-template-columns: 1fr; }              ← …события выстраиваются в одну колонку
  .event .thumb { order: -1; }                        ← …а фото переходит над текстом
}
```

Медиазапросы (`@media`) — это правила «если… то…» в зависимости от размера экрана. Но «самоподстраивающаяся» сетка из первого примера часто избавляет от них: подвал устроен так же (`minmax(180px, 1fr)`).

Меню на телефоне

На маленьком экране меню прячется за кнопкой «Меню». Код — в `src/layouts/Base.astro` (кнопка `.menu-btn`), стиль — в `global.css`.

Детали, которые делают «профессионально»

- `text-wrap: balance`: у заголовков в две строки строки получаются примерно одной длины.
- `@media (prefers-reduced-motion)`: анимация отключается для тех, кому она мешает.
- Максимальная ширина (`--max: 1440px`), чтобы текст не растягивался по гигантскому экрану.

> **ПОПРОБОВАТЬ**
>
> F12 → значок телефона/планшета («режим устройства») → выберите «iPhone»: посмотрите, как сайт перестраивается. Понемногу расширяйте окно и следите, когда меняются колонки.

9 · Фото и видео

Изображения часто составляют 80 % веса страницы. Фото с телефона весит 4–8 МБ; на сайте то же фото весит 50–200 КБ. Как так?

Три автоматические оптимизации

1. **При загрузке в админке**: фото уменьшается максимум до 2000 px и превращается в **WebP**, формат легче JPEG (настройка `transformations` в `public/admin/config.yml`).
2. **При сборке сайта**: Astro делает **5 размеров** каждого фото (480, 800, 1200, 1600, 1920 px). Файл: `src/components/Img.astro`.
3. **В браузере**: благодаря атрибутам `srcset` и `sizes` телефон скачивает маленькую версию, большой экран — большую. А `loading="lazy"` значит, что фото скачивается, только когда вы до него прокручиваете.

Альтернативный текст

У каждого изображения есть `alt`: описание, которое зачитывается незрячим людям и используется Google. Например, на главной: «Финальное расслабление в шале Evergreen Project» (меняется в админке).

Видео

- **YouTube / Vimeo** (рекомендуется): вставляется ссылка; компонент `Video.astro` показывает только миниатюру и загружает плеер по клику (версия «youtube-nocookie», без cookies) → страница быстрая и бережная.
- **Файл MP4**: робот GitHub (`.github/workflows/compress-videos.yml`) пережимает его инструментом **ffmpeg** (формат H.264, максимум 1280 px).

Полноэкранный просмотр

В `src/pages/[lang]/gallery.astro` клик по фото открывает его крупно во встроенном HTML-элементе `<dialog>` — без внешних библиотек: браузер сам управляет окном, клавишей Esc и фокусом.

10 · Сайт на 8 языках

Греческий (язык по умолчанию), английский, французский, испанский, итальянский, русский, немецкий, португальский. Три механизма:

1. **Адреса** начинаются с языка: `/el/`, `/ru/`... Папка `[lang]` в `src/pages/` собирает каждую страницу на каждом языке. Список языков — в `src/i18n.ts`.
2. **Содержимое**: в каждом YAML-файле есть часть для каждого языка (глава 6).
3. **Мелкие тексты** («Меню», «Подробнее», «Добавить в календарь»...): `src/content/settings/ui.yml`, 96 фраз × 8 языков.

«Запасной вариант» (fallback)

Если фраза не переведена на какой-то язык, сайт показывает английский вариант, а если нет и его — греческий. Пустых мест не бывает. Код: функция `localize` в `src/lib.ts`.

Сообщить Google о переводах

В `<head>` каждой страницы (`Base.astro`) теги `<link rel="alternate" hreflang="ru">` указывают версии на других языках: русскоязычный посетитель попадает на русскую страницу.

> **ПОСМОТРЕТЬ**
>
> На сайте переключатель языка справа вверху: он оставляет ту же страницу и меняет только префикс (`/ru/contact/` → `/de/contact/`).

11 · Функции, одна за другой

Для каждой функции: что она делает, какой инструмент или сервис использует и какой файл открыть.

Функция	Как сделано	Где посмотреть
Календарь на месяц	Небольшой скрипт на JavaScript рисует месяц и закрашивает занятые дни; прошедшие события исчезают сами (сравнение с сегодняшней датой).	`src/pages/[lang]/calendar.astro`, `src/components/EventList.astro`
«Добавить в календарь» (Google)	Простая ссылка на Google Календарь с названием, датами и часовым поясом «Europe/Athens»: Google переводит во время посетителя.	`src/calendar.ts` (функция `googleUrl`)
Apple / Outlook (.ics)	Файл `.ics`: стандартный текстовый формат (RFC 5545), который понимают все календари. Время переводится во всемирное с помощью встроенного в JavaScript инструмента `Intl`.	`src/calendar.ts` (функция `ics`), `src/pages/events/[id].ics.ts`
Подписка на календарь	Файл `/calendar.ics` со всеми событиями; календарь посетителя регулярно его перечитывает.	`src/pages/calendar.ics.ts`
Карта	Карта **OpenStreetMap** (свободная и бесплатная) внутри `<iframe>` плюс ссылка «Открыть в картах» на Google Maps.	`src/pages/[lang]/[page].astro` (часть «Контакты»)
Форма обратной связи	Сервис **Web3Forms**: форма отправляет поля на их сервер, а он пересылает их по e-mail. Невидимое поле ловит роботов («honeypot»).	тот же файл, `<form action="https://api.web3forms.com/submit">`
Пожертвование / IBAN	IBAN группами по 4 символа + кнопка, которая его копирует (API **Clipboard** браузера).	тот же файл, блок `donate` + `<script>` внизу
Истории	Форма → маленький сервер (Worker, глава 13) → сохраняется «неопубликованной» → администратор одобряет.	`src/pages/[lang]/stories.astro`, `worker/index.js`
Галерея	Альбомы «мозаикой» на CSS, просмотр в `<dialog>`.	`src/pages/[lang]/gallery.astro`
Полоса объявления	Если поле «Ανακοίνωση» в админке заполнено, вверху всех страниц появляется оранжевая полоса.	`Base.astro` (`site.announcement`)
Ссылка на студии йоги	Внешняя ссылка (без дублирования: занятия и цены остаются на их сайте).	`src/content/settings/site.yml`
Оформление текста	Длинные тексты пишутся на **Markdown** (`**жирный**`, `- список`, `[ссылка](адрес)`), который пакет **marked** превращает в HTML.	`src/lib.ts` (функция `md`)

12 · Панель администратора (без кода)

Катерина и Костас не будут редактировать YAML-файлы. Поэтому у них есть **интерфейс** по адресу `/admin/`: формы, кнопки, перетаскивание фото. Это **CMS** (Content Management System — система управления содержимым).

Выбранная CMS: Sveltia CMS

- Бесплатная, с открытым кодом, работает целиком в браузере.
- Читает и записывает YAML-файлы репозитория напрямую: **никакой базы данных**.
- Показывает 8 языков рядом и сжимает фото при загрузке.

Как она настроена

Всего два файла:

- `public/admin/index.html`: загружает программу Sveltia (зафиксированная версия + отпечаток безопасности, см. главу 16).
- `public/admin/config.yml`: описывает **каждую форму**. Для каждого поля: имя, подпись, тип («виджет»).

```
- { name: start, label: Έναρξη · Start date, widget: datetime, type: date }   ← календарь для выбора даты
- { name: price, label: Τιμή · Price, widget: string, i18n: true }          ← текстовое поле, переводимое
- { name: image, label: Φωτογραφία · Photo, widget: image }                ← кнопка загрузки фото
- { name: published, label: Published, widget: boolean }                   ← флажок
```

Что происходит после нажатия «Save»

Админка: Save → YAML-файл меняется в репозитории → Робот пересобирает сайт → Онлайн через ~2 мин

Каждое сохранение — это «коммит»: всегда можно вернуться к прежней версии.

Помощь в написании

Рядом с редактируемым полем появляется кнопка **«✓ Correct»**: она предлагает исправленный вариант (орфография, грамматика, пунктуация), который можно принять или нет. А при каждом сохранении новые или изменённые греческие тексты **автоматически переводятся** на остальные 7 языков. И то и другое проходит через Worker (глава 13), который обращается к сервису автоматической проверки и перевода. Код: `public/admin/index.html` (кнопка) и `worker/ai.js`.

> **ПОСМОТРЕТЬ**
>
> `docs/GUIDE-ADMIN.md`: инструкция, написанная для администраторов (по-гречески и по-английски). Хороший пример документации для нетехнических людей.

13 · Маленький сервер: Worker

Статический сайт не может хранить секреты или принимать данные (на сервере не работает никакая программа). Поэтому для нескольких задач мы добавляем **крошечный** бесплатный **сервер** в **Cloudflare**: **Worker** (функция на JavaScript, которая запускается, когда к ней обращаются).

Задача	Что делает Worker
Вход администраторов	Показывает форму «e-mail + пароль», проверяет её и выдаёт «пропуск» на сессию (на 30 дней), подписанный секретным ключом (подпись HMAC), чтобы его нельзя было подделать.
Сохранения в админке	Передаёт запросы админки в хранилище, добавляя настоящий ключ доступа, который знает только Worker.
Помощь в написании	Исправляет текст по запросу; переводит каждое сохранение на 7 языков перед тем, как его записать.
Истории	Принимает форму, отсеивает спам, проверяет, что фото действительно изображение, сохраняет историю «неопубликованной» и уведомляет команду.

«Секреты» (пароли администраторов, ключ доступа) хранятся в Cloudflare в зашифрованном виде — **никогда** в коде репозитория.

> **ПОСМОТРЕТЬ**
>
> `worker/index.js` (с подробными комментариями, разделён на части: вход, передача, истории) · `worker/ai.js` (помощь в написании) · `worker/wrangler.toml` (настройки) · `worker/README.md` (установка шаг за шагом). Инструмент **wrangler** (`npx wrangler deploy`) отправляет Worker в Cloudflare.

API: слово, которое нужно знать

API — это «дверь», которую программа открывает другим программам: отправляешь запрос на адрес — получаешь ответ. Сайт пользуется несколькими:

API	Для чего
API GitHub	Читать / записывать файлы с содержимым (через Worker)
Сервис перевода / проверки	Переводить и исправлять тексты админки (через Worker)
Web3Forms	Отправлять форму обратной связи по e-mail
Cloudflare Turnstile (по желанию)	Защита от роботов, бережная альтернатива «капче»
API браузера	Буфер обмена (копировать IBAN), `Intl` (даты и часовые пояса), `<dialog>`
Google Календарь (ссылка)	Добавить событие в свой календарь

14 · Чтобы Google находил сайт (SEO)

SEO (поисковая оптимизация) — это помощь поисковикам в том, чтобы они понимали сайт и хорошо его показывали.

Элемент	Роль	Где
`<title>` и `<meta name="description">`	Синий заголовок и две строки текста в выдаче Google.	`Base.astro`
Open Graph (`og:image`…)	Превью (фото + заголовок), когда ссылкой делятся в Facebook, WhatsApp…	`Base.astro`
Карта сайта (`sitemap-index.xml`)	Список всех страниц для Google; создаётся пакетом **@astrojs/sitemap**.	`astro.config.mjs`
`robots.txt`	Сообщает роботам, что можно посещать (админку — нельзя).	`src/pages/robots.txt.ts`
Структурированные данные (JSON-LD)	Карточка для машин: «это НКО, вот её адрес, телефон» (словарь **schema.org**).	`Base.astro` (`ld`)
`hreflang`	Переводы каждой страницы.	`Base.astro`
Понятные адреса	`/ru/contact/` вместо `/page?id=37`.	имена файлов в `src/pages/`

ПОПРОБОВАТЬ

Вставьте адрес сайта в *opengraph.xyz*, чтобы увидеть превью при репосте, или найдите «Проверку расширенных результатов» Google, чтобы увидеть карточку JSON-LD.

15 · Доступность: сайт для всех

Примерно у каждого пятого человека есть инвалидность (зрение, подвижность, слух, внимание…). Доступный сайт удобнее для всех, и Google ранжирует его выше.

- **Альтернативные тексты** у изображений (`alt`).
- Достаточный **контраст**: светло-серый текст на кремовом фоне многие не могут прочитать. Подпись в подвале сделали темнее именно поэтому.
- **Заголовки по порядку** (`h1`, затем `h2`…): программы экранного доступа используют их как оглавление.
- Скрытая **ссылка «Перейти к содержимому»**, видимая при работе с клавиатуры (`.skip` в `Base.astro`): не нужно проходить всё меню.
- **Навигация с клавиатуры**: видимая рамка вокруг выбранного элемента (`:focus-visible` в `global.css`).
- **Понятные названия** у кнопок и ссылок без текста (`aria-label`): например, у фото-ссылок на главной.
- Учёт настройки телефона «уменьшить движение».

ПОПРОБОВАТЬ

На сайте несколько раз нажмите клавишу **Tab**: всегда должно быть видно, где вы находитесь. Затем F12 → вкладка **Lighthouse** → «Accessibility» → Analyze.

16 · Безопасность

Риск	Защита на этом сайте	Где
Злоумышленник внедряет скрипт в страницу	**CSP** (Content-Security-Policy): белый список того, что странице можно загружать; всё остальное браузер блокирует.	`astro.config.mjs` (`security.csp`)
Внешний сервис (CMS) незаметно подменяют	**Зафиксированная версия + SRI**: отпечаток файла; если изменится хоть один байт, браузер откажется его запускать.	`public/admin/index.html` (`integrity="sha384-…"`)
Утечка паролей / ключей	Секреты только в Cloudflare, никогда в репозитории; локальные файлы с секретами Git игнорирует.	`.gitignore`, `worker/README.md`
Подбор паролей, спам	Не больше 3 попыток в минуту с одного адреса, поле-ловушка, минимальное время заполнения, Turnstile по желанию.	`worker/index.js`, `wrangler.toml`
Загрузка поддельных файлов	Проверка первых байтов («подписи»): JPEG всегда начинается с `FF D8 FF`.	`worker/index.js` (`looksLike`)
Перехват соединения	HTTPS везде (замочек).	хостинг
Сайт показывают внутри рамки-ловушки	Заголовок `X-Frame-Options`.	`public/_headers`
Слежка за посетителями	Никаких cookies и счётчиков статистики, шрифты и видео без трекеров.	страница «Конфиденциальность»

> **СКАЗАТЬ ВСЛУХ**
>
> «Лучшая защита — когда почти нечего защищать: у статического сайта нет базы данных и паролей, которые можно украсть. Немногие секреты хранятся в одном месте».

17 · Скорость (производительность)

Медленный сайт отпугивает людей (и Google его понижает). Рецепты, которые здесь применены:

- **Страницы уже готовы** (статические): во время визита ничего не нужно вычислять.
- **Изображения**: WebP, несколько размеров, отложенная загрузка (глава 9). А первое большое фото, наоборот, грузится в первую очередь (`fetchpriority="high"`).
- **Очень мало JavaScript**: только небольшие скрипты там, где они нужны. Astro по умолчанию не отправляет JavaScript.
- **Шрифты хранятся на самом сайте**, а не в Google Fonts.
- **Видео загружаются по клику**.
- **Кэш**: файлы, в имени которых есть отпечаток (папка `_astro/`), браузер хранит год (`public/_headers`).

Результат, измеренный **Lighthouse** (встроен в Chrome, вкладка «Lighthouse») на мобильном: производительность 98–99/100, главная страница ≈ 256 КБ.

Показатели, которые нужно знать

Показатель	Что значит	Цель
LCP	Время до показа самого крупного элемента (часто главного фото)	< 2,5 с
CLS	«Прыгает» ли страница во время загрузки?	< 0,1
INP / TBT	Быстро ли страница отвечает на клики?	как можно меньше

18 · Проверка сайта

Автоматические тесты

Небольшие программы проверяют, что код делает то, что должен. Здесь — встроенным в Node инструментом (`node:test`), запуск командой `npm test`:

- `tests/calendar.test.ts`: «10:00 в Афинах в декабре = 8:00 по всемирному времени, но 7:00 в июле (летнее время)», «файл .ics составлен правильно»...
- `tests/worker.test.ts`: «неверный пароль отклоняется», «поддельный файл изображения отклоняется», «секретный ключ никогда не попадает в браузер»...
- `tests/ai.test.ts`: «переводится только новый или изменённый греческий текст», «перевод, написанный вручную, сохраняется»...

Ручные проверки (после каждого крупного изменения)

Что	Как
Отображение	На настоящем телефоне + режим устройства в инструментах разработчика; Chrome, Safari, Firefox.
Битые ссылки	Прокликать всё или воспользоваться инструментом проверки ссылок.
Доступность	Lighthouse, расширение **axe DevTools**, навигация с клавиатуры.
Ошибки	F12 → вкладка «Console»: никаких красных строк.
Содержимое	Перечитать на каждом языке (в идеале — носителями языка).

19 · Публикация: хостинг, домен, DNS

Хостинг

Папка `dist/`, собранная командой `npm run build`, выкладывается на **хостинг**. Для статического сайта хватает бесплатных сервисов: **GitHub Pages** (используется сейчас), **Cloudflare Pages** (запланирован дальше), Netlify…

Робот публикации

`.github/workflows/deploy.yml` — это рецепт, который GitHub автоматически выполняет при каждом изменении в `main`: установить Node → `npm ci` → `npm run build` → опубликовать `dist/`. Это **непрерывная интеграция и доставка** (CI/CD): никому не нужно ничего «загружать» вручную.

Доменное имя

Имя вроде `evergreenproject.gr` покупается (≈ 10–15 € в год) у **регистратора** (Papaki, DNHOST, OVH…). Его арендуют на год, а не владеют им: следите за продлением!

DNS: телефонная книга интернета

DNS превращает имя (`evergreenproject.gr`) в адрес сервера. В него добавляют «записи»:

- **A**: «это имя указывает на этот сервер (IP-адрес)».
- **CNAME**: «это имя — псевдоним другого имени» (например, `www` → хостинг).
- **MX**: «почта этого домена идёт сюда».

> **ПОСМОТРЕТЬ**
>
> В `README.md` — разделы о собственном домене и об уходе с GitHub: конкретные шаги, чтобы подключить домен и сменить хостинг.

20 · Жизненный цикл изменения

Для разработчика (изменить дизайн или функцию)

1. Создать **ветку** (рабочую копию).
2. Изменить файлы в VS Code; проверять вживую через `npm run dev`.
3. Запустить тесты (`npm test`) и сборку (`npm run build`).
4. Сделать **коммит** с понятным сообщением, **push** на GitHub.
5. Открыть **pull request**, проверить и **слить** в `main`.
6. Робот публикует: через ~2 минуты это видят все.

Для администратора (изменить текст, дату, фото)

1. Открыть `/admin/`, войти.
2. Изменить, нажать **Save**.
3. Через ~2 минуты всё на сайте. Ничего не нужно устанавливать.

> **РЕАЛЬНЫЙ ПРИМЕР**
>
> В истории репозитория («Commits») найдите «Inhabitants: new photo of our dogs»: новое фото в `src/assets/uploads/` и одна изменённая строка в `src/content/inhabitants/dogs.yml` — ровно то, что делает админка при замене фото.

21 · С чего начать учиться

Рекомендуемый путь (около 3 месяцев по несколько часов в неделю)

Этап	Цель	Бесплатные ресурсы
1. HTML	Сделать страницу с заголовками, текстом, картинками, ссылками	MDN Web Docs («Изучение веб-разработки»), freeCodeCamp
2. CSS	Цвета, шрифты, потом Flexbox и Grid	Игры **Flexbox Froggy** и **Grid Garden**, web.dev/learn/css
3. Адаптивность	Страница, которая подстраивается под телефон	web.dev/learn/design
4. Git и GitHub	Сохранять и публиковать свою работу	GitHub Skills, «Learn Git Branching»
5. JavaScript	Добавить немного интерактивности (кнопка, меню)	learn.javascript.ru, freeCodeCamp
6. Генератор	Пересобрать небольшой сайт на Astro	docs.astro.build (учебник «Создайте блог»)
7. Публикация	Бесплатно выложить свой сайт в интернет	GitHub Pages, Cloudflare Pages

Небольшие упражнения на этом сайте (без риска, на своём компьютере)

1. Изменить цвет `--clay` в `src/styles/global.css` и посмотреть, что будет.
2. Добавить пробное событие: скопировать файл из `src/content/events/`, поменять даты и название.
3. Изменить текст кнопки в `src/content/settings/ui.yml`.
4. Добавить обитателя (животное!) в `src/content/inhabitants/`.
5. Прочитать `src/components/Story.astro` и попробовать создать свой компонент.

> **ПОСЛЕДНИЙ СОВЕТ**
>
> «Учатся не чтением, а тем, что ломают и чинят. С Git ничего никогда не ломается по-настоящему: всегда можно вернуться назад».

22 · Словарь

API	«Дверь», которую сервис открывает другим программам.
Сборка (build)	Изготовление финального сайта (папка `dist/`).
Кэш	Копия, которую хранит браузер, чтобы не скачивать заново.
CI/CD	Роботы, которые автоматически проверяют и публикуют.
CMS	Интерфейс для управления содержимым без программирования (здесь Sveltia, по адресу `/admin/`).
Коммит	Изменение, сохранённое в истории, с сообщением.
Компонент	Повторно используемый блок страницы (`Img`, `Story`…).
CSP	Белый список безопасности: что странице разрешено загружать.
DNS	Справочник, который превращает доменное имя в адрес сервера.
Репозиторий (repo)	Папка проекта со всей историей (на GitHub).
Фронтенд / бэкенд	То, что работает у посетителя / на сервере.
Хостинг	Сервис, который держит файлы сайта в интернете круглосуточно.
i18n	«Интернационализация» (i + 18 букв + n): работа с языками.
Layout	Общий каркас всех страниц.
Markdown	Простой способ писать оформленный текст (`**жирный**`).
npm / пакет	Магазин инструментов JavaScript / инструмент, скачанный оттуда.
Pull request	Запрос на слияние ветки, с проверкой.
Адаптивный (responsive)	Подстраивающийся под любой размер экрана.
SEO	Чтобы Google находил и хорошо показывал сайт.
SRI	Отпечаток, гарантирующий, что внешний файл не подменён.
Статический	Страницы, собранные заранее, а не при каждом посещении.
URL	Адрес страницы.
WebP	Современный формат изображений, легче JPEG.
Worker	Маленькая серверная функция в Cloudflare.
YAML	Читаемый текстовый формат («ключ: значение»), в котором хранится содержимое.